Création de la cible `price_category`

Module M2 — Création de la cible et analyse des classes

Jeu de données : King County (Washington), 21 428 logements · Rapport du 4 novembre 2026 · Dépôt GitHub du projet, dossier `Target_creation/`

En bref. Chaque logement reçoit l'une de quatre classes de prix, calculée en comparant son prix au pied carré à la médiane de logements comparables (même code postal, même tranche de surface). Seuils du ratio : 0,85 / 1,15 / 1,40. Répartition obtenue : 19,0 % / 58,6 % / 16,0 % / 6,4 %. La classe majoritaire sert de référence : un classifieur qui prédit toujours « prix raisonnable » a 58,6 % d'exactitude.

1. Contexte et objectif

Le jeu de données contient le prix demandé de chaque logement, mais aucune information indiquant si ce prix est raisonnable. Un classifieur supervisé a besoin d'une réponse connue pour chaque exemple. La mission du module M2 est donc de construire cette réponse, la cible `price_category`, en comparant le prix d'un logement à celui de logements comparables et non au prix global du marché.

Classe	Nom	Signification
0	Cheaper than average	Moins cher que la moyenne
1	Reasonable price	Prix raisonnable
2	Over priced	Trop cher
3	Too over priced	Beaucoup trop cher

Le prix demandé reste une variable d'entrée du modèle : l'application web recevra le prix saisi par l'utilisateur et dira s'il est attractif ou trop élevé.

2. Données d'entrée

Le module lit le jeu nettoyé produit par M1 : `Data_cleaning/data/processed/house_clean.csv` (21 428 lignes, 15 colonnes : `price`, `bedrooms`, `bathrooms`, `sqft_living`, `sqft_lot`, `floors`, `waterfront`, `view`, `condition`, `grade`, `sqft_above`, `sqft_basement`, `yr_built`, `zipcode`, `renovated`). Aucune valeur manquante, aucun prix ni surface nul. Aucune ligne n'a été supprimée ni modifiée par M2.

Le notebook retrouve ce fichier tout seul, quel que soit le dossier d'où il est lancé, et s'arrête avec un message clair s'il est introuvable. `zipcode` est lu comme du texte : c'est un identifiant de zone, pas un nombre.

3. Méthode

1. Prix comparable : prix au pied carré = `price` / `sqft_living`.
2. Groupes de comparaison : même code postal (70 zones) et même tranche de surface (3 tranches de même effectif, obtenues avec `pd.qcut` : 370–1 584, 1 590–2 310 et 2 311–13 540 pieds carrés). On obtient 210 groupes de 3 à 343 logements (médiane : 85).
3. Médiane du groupe : médiane du prix au pied carré de chaque groupe. Si un groupe compte moins de 30 logements (15 groupes, soit 254 logements, 1,2 %), on utilise la médiane de son code postal. Aucun code postal ne compte moins de 30 logements (le plus petit en compte 49) : le repli sur la médiane globale n'a jamais été nécessaire.
4. Ratio = prix au pied carré du logement / médiane de son groupe. Un ratio de 1,00 signifie un prix au niveau de ses semblables.
5. Classes : seuils 0,85 / 1,15 / 1,40 appliqués au ratio (`np.digitize`).

4. Choix des groupes de comparaison

Les groupes ont été choisis à partir de mesures sur les données, pas par intuition.

Question	Mesure
Le code postal explique-t-il le prix au pied carré ?	Oui : environ 61 % de la variance du logarithme du prix au pied carré à lui seul.
La surface apporte-t-elle quelque chose ?	Oui : 3 tranches portent ce chiffre à environ 67 %. Le prix au pied carré médian est de 282 pour les logements de 370 à 1 584 pieds carrés, 227 pour ceux de 1 590 à 2 310 et 235 au-delà : la taille joue surtout pour les petites maisons.
Pourquoi pas 4 tranches ?	La part de logements dans un groupe de moins de 30 passerait de 1,2 % à 3,7 % pour un gain d'environ 1 point de variance expliquée.
Pourquoi pas les chambres ?	951 groupes, dont 25,3 % des logements dans un groupe de moins de 30 : trop petits pour une médiane fiable. Le gain apparent (environ 70 %) est de toute façon mécanique, car la variance expliquée croît avec le nombre de groupes.

5. Choix des seuils

La distribution du ratio est centrée sur 1,00 (médiane exacte), avec une queue à droite : 5 % des logements sont sous 0,71 et 5 % au-dessus de 1,46. La moitié centrale des ratios se situe entre 0,885 et 1,129.

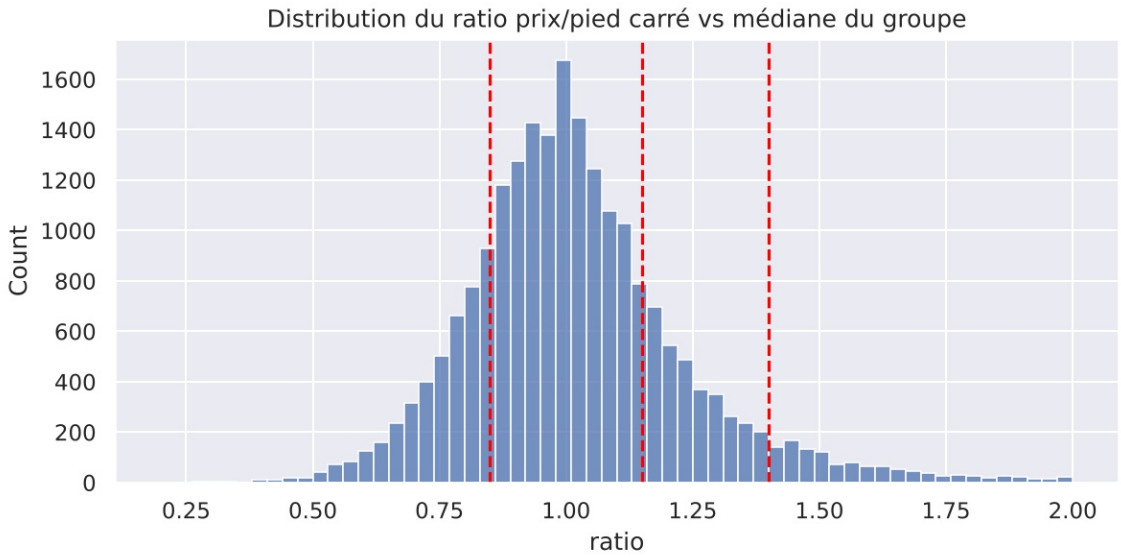


Figure 1 — Distribution du ratio prix au pied carré / médiane du groupe. Les traits rouges marquent les seuils 0,85, 1,15 et 1,40.

0,85 et 1,15 : un prix est jugé raisonnable lorsqu'il s'écarte d'au plus 15 % de la médiane de ses pairs. La règle est simple à expliquer et à défendre. Ces seuils correspondent aux percentiles 19,0 et 77,6 du ratio.

1,40 : il sépare « trop cher » de « beaucoup trop cher » et correspond au 93,6^{e} percentile. Il laisse 1 367 logements à la classe 3, assez pour qu'un modèle puisse l'apprendre.

Ces seuils sont un choix de conception, pas une vérité mesurée. L'analyse de sensibilité ci-dessous montre leur effet sur la répartition des quatre classes.

Seuils	Classe 0	Classe 1	Classe 2	Classe 3
0,85 / 1,15 / 1,40 (retenus)	19,0 %	58,6 %	16,0 %	6,4 %
0,80 / 1,20 / 1,50	12,5 %	70,6 %	12,8 %	4,1 %
0,85 / 1,15 / 1,30	19,0 %	58,6 %	12,2 %	10,2 %
0,90 / 1,10 / 1,30	27,9 %	42,6 %	19,4 %	10,2 %

6. Résultat : répartition des classes

Classe	Nom	Logements	Part
0	Cheaper than average	4 076	19,0 %
1	Reasonable price	12 559	58,6 %
2	Over priced	3 426	16,0 %
3	Too over priced	1 367	6,4 %

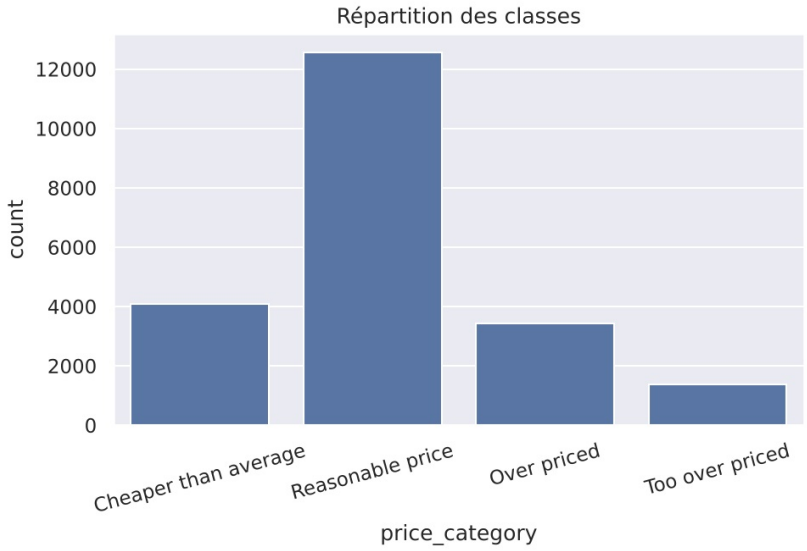


Figure 2 — Répartition des quatre classes.

La classe 3 est minoritaire mais pas rare. Un classifieur qui prédit toujours la classe majoritaire (Zero-R) obtient 58,6 % d'exactitude : c'est la référence à dépasser. Comme les classes sont déséquilibrées, le F1 macro est plus informatif que l'exactitude seule.

7. Vérifications de cohérence

Deux tests sur un logement de référence (ratio actuel 1,14, classe 1), avec la médiane du groupe supposée fixe :

Test 1 — seul le prix demandé change		
Prix	**Ratio**	**Classe**
× 0,6	0,69	0
× 0,8	0,91	1
× 1,0	1,14	1
× 1,2	1,37	2
× 1,5	1,72	3
× 2,0	2,29	3

Test 2 — même prix, surface différente		
Surface	**Ratio**	**Classe**
× 0,7	1,63	3
× 1,0	1,14	1
× 1,3	0,88	1
× 1,6	0,71	0

Test 1 : la classe change bien avec le seul prix (de 0 à 3) et ne baisse jamais quand le prix augmente. Le notebook le vérifie par des `assert`.
Test 2 : à prix égal, une surface plus petite donne un prix au pied carré plus élevé, donc une classe plus chère ; une plus grande surface donne une classe moins chère ou égale.
Limite : changer la surface peut changer de tranche, donc de groupe. Le test vérifie le sens de la règle, il ne simule pas une prédiction du modèle.
Aucune colonne auxiliaire (ratio, médiane du groupe, prix au pied carré, tranche de surface) n'est présente dans le fichier étiqueté : une assertion du notebook le vérifie.

8. Exemples pour la démonstration

Deux logements par classe, choisis de façon déterministe : surface et prix dans la zone centrale du jeu de données, ratio proche de celui d'un logement typique de la classe, codes postaux différents. Ce sont des exemples clairs, loin des frontières entre classes.

Classe	Prix demandé	Surface (pi²)	Code postal	Prix / pi²	Médiane du groupe	Ratio
0 · Cheaper than average	290 500 $	2 510	98055	116	150	0,77
0 · Cheaper than average	398 000 $	1 720	98136	231	300	0,77
1 · Reasonable price	645 000 $	2 250	98006	287	288	0,99
1 · Reasonable price	600 000 $	1 520	98115	395	397	0,99
2 · Over priced	509 990 $	1 440	98118	354	287	1,23
2 · Over priced	856 000 $	1 480	98112	578	469	1,23
3 · Too over priced	520 000 $	1 940	98038	268	171	1,57
3 · Too over priced	689 800 $	2 390	98133	289	184	1,57

Idée de démonstration : saisir un même logement avec des prix de plus en plus élevés et montrer la classe qui monte de « moins cher que la moyenne » à « beaucoup trop cher ».

9. Limites

> **Fuite par construction.** Le modèle reçoit `price`, `sqft_living` et `zipcode`, qui servent aussi à construire la cible. Un score élevé reflète donc en partie la règle que nous avons écrite, et non une compréhension du marché.

Contrôle indicatif (notebook, section 9 bis) : un arbre de profondeur 12, entraîné sur le découpage 80/20 stratifié du projet (`random_state=42`). Ce n'est pas le modèle final, qui reste le travail de M4.

Modèle	Exactitude	F1 macro
Zero-R (classe majoritaire)	0,586	0,185
Arbre, toutes variables (avec `price`)	0,669	0,543
Arbre, sans la colonne `price`	0,588	0,398

Sans le prix, l'exactitude (0,588) retombe au niveau de Zero-R (0,586), mais le F1 macro reste supérieur à celui de Zero-R (0,398 contre 0,185) : le prix porte l'essentiel du signal, mais pas tout. Ce contrôle code `zipcode` comme un entier ; ses valeurs sont indicatives, pas une borne.

> **La qualité du bien n'entre pas dans les groupes.** Les logements en bord d'eau (163 logements, 0,76 %) ont un ratio médian de 2,03, ceux avec la meilleure vue de 1,56, et le ratio médian monte aux deux extrémités de la note de construction (1,88 pour le grade 3, 1,28 pour le grade 12). Ces logements sont donc souvent classés « trop chers » même quand leur prix se justifie.

- La médiane d'un groupe inclut le logement évalué lui-même ; l'effet est faible avec des groupes de plusieurs dizaines de logements.
- Le choix des seuils influence directement la taille de la classe 3 (voir la sensibilité, section 5).
- Données signalées à M1 et M3, sans rien supprimer : 5 lignes dupliquées, 6 logements à 0 chambre et 3 à 0 salle de bain.

10. Consignes pour les autres membres

Pour	Consigne
M3, M4	Charger le fichier avec `pd.read_csv("house_labeled.csv", dtype={"zipcode": str})` puis encoder `zipcode` en one-hot, sans l'échelonner comme une variable numérique. Ne jamais utiliser `price_per_sqft`, `size_bucket`, `peer_median`, `peer_level` ni `ratio` comme variables d'entrée. Utiliser `stratify=y`, regarder le F1 macro et tester `class_weight="balanced"` (classe 3 : 6,4 %).
M5, M6	`feature_options.json` donne les valeurs de `zipcode` (70 codes, type texte) et les plages numériques. `waterfront` et `renovated` sont binaires : à présenter comme des interrupteurs. Le formulaire doit envoyer `zipcode` en texte.

11. Livrables et reproduction

Tous les fichiers sont dans le dossier `Target_creation/` du dépôt.

Fichier	Contenu
`notebooks/M2_target_creation.ipynb`	Code complet, avec ses résultats
`data/labeled/house_labeled.csv`	Variables d'origine et `price_category`
`data/labeled/target_helpers.csv`	Colonnes auxiliaires (à ne jamais donner au modèle)
`data/labeled/target_definition.md`	Documentation de la cible
`data/labeled/feature_options.json`	Valeurs et plages pour M3, M5 et M6
`data/labeled/example_houses.csv`, `example_houses_explained.csv`	Exemples pour la démonstration
`data/labeled/price_dependency_check.csv`	Contrôle indicatif de la dépendance au prix
`docs/figures/target/`	Figures 1 et 2
`docs/report/`	Ce rapport (PDF) et sa version Markdown

Reproduire : ouvrir `notebooks/M2_target_creation.ipynb` depuis le dépôt et choisir *Restart & Run All*.
Versions utilisées : Python 3.12.10, pandas 2.3.3, numpy 2.4.4, scikit-learn 1.9.1, matplotlib 3.10.8, seaborn 0.13.2.
Les résultats sont identiques d'une exécution à l'autre (`random_state=42`).